

Introdução às Funções em Python

Automatizando Tarefas como Administradores de Redes

Prof. Reginaldo Fernandes
Programação de Computadores

Instituto Federal do Ceará – Campus Tauá
Curso Técnico em Redes de Computadores

5 de agosto de 2026

Sumário da Aula

- 1 Introdução às Funções
- 2 Funções Sem Parâmetros e Sem Retorno
- 3 Funções Com Parâmetros e Sem Retorno
- 4 Funções Sem Parâmetros e Com Retorno
- 5 Funções Com Parâmetros e Com Retorno
- 6 Resumo e Atividade Prática

O Problema: Repetição Ineficiente

Imagine que você é o administrador de rede do IFCE Campus Tauá. Você precisa exibir uma mensagem de alerta no terminal do servidor toda vez que:

- Um usuário fizer login no sistema.
- Uma conexão cair.
- Um backup for concluído.

Sem funções, você teria que copiar e colar o mesmo código várias vezes:

Código Repetitivo (Sem Funções)

```
print("=====  
print("ALERTA DE SEGURANCA DA REDE "  
print("=====
```

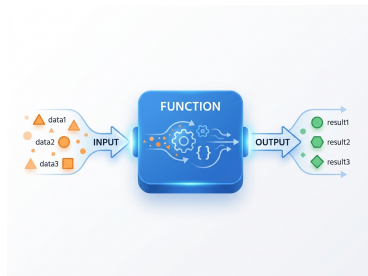
Problema: Se você quiser mudar o texto para "ALERTA DE SISTEMA", terá que alterar em todas as partes do código!

O que é uma Função?

Na programação, uma **função** é um bloco de código que executa uma tarefa específica e pode ser reutilizado a qualquer momento.

Analogias de Redes:

- **Um Comando do Switch:** Quando você digita ping, há um código interno complexo rodando, mas você só vê o nome ping.
- **Servidor DHCP:** Recebe o endereço físico (MAC) e **retorna** um IP configurado.



Entrada (IP/Dados) → Processamento → Saída (Resultado)

Como Definir uma Função em Python

Para criar uma função, usamos a palavra-chave **def** (de *define* ou definir), seguida pelo nome da função, parênteses e dois pontos.

```
1 def nome_da_funcao(parametro1, parametro2):  
2     # Corpo da funcao (o que ela faz)  
3     #Codigo indentado  
4     return resultado # Opcional
```

Regras importantes:

- O nome deve ser explicativo (ex: `verificar_conexao`).
- O bloco de código interno **deve estar indentado** (com 4 espaços).
- Para executar a função, nós a “chamamos” pelo nome: `nome_da_funcao()`.

1. Sem Parâmetros e Sem Retorno

Essas funções executam tarefas fixas. Elas não precisam de nenhuma informação externa para rodar (sem parâmetros) e não devolvem valores para o programa principal (sem retorno).

Analogia de Redes:

- É como o comando `clear` no terminal do Linux ou `show clock` no switch Cisco.
- Você digita o comando, ele executa uma ação na tela, mas não te dá nenhum dado para guardar em uma variável.

Sintaxe:

```
1 def minha_funcao():  
2     # Executa acao direta  
3     print("Acao executada!")
```

Exemplo 1: Banner da Rede do IFCE

Vamos criar uma função para exibir o cabeçalho de boas-vindas do terminal da rede do campus.

```
1 def exibir_banner():
2     # Exibe cabeçalho estatico no console da rede
3     print("+" + "-"*40 + "+")
4     print("|   SISTEMA DE GERENCIAMENTO DE REDES   |")
5     print("|               IFCE CAMPUS TAUA               |")
6     print("+" + "-"*40 + "+")
7
8 # Como utilizar a funcao no script principal:
9 exibir_banner()
```

Resultado no Terminal:

Saída

```
+-----+
| SISTEMA DE GERENCIAMENTO DE REDES |
| IFCE CAMPUS TAUA |
+-----+
```

Exemplo 2: Alerta de Reinicialização

Função para exibir um aviso sonoro (usando caracteres de alerta) e aviso de texto que o roteador principal será reiniciado.

```
1 def avisar_reboot():
2     # Emite aviso sobre desligamento do roteador
3     print("[ATENCAO] O Roteador Central sera reiniciado em 5 minutos!")
4     print("Por favor, salvem as configuracoes pendentes.")
5     print("Todas as conexoes ativas serao interrompidas.")
6
7 # Simulando o agendamento de manutencao:
8 print("Iniciando rotina de manutencao...")
9 avisar_reboot()
```

Saída:

Saída

```
Iniciando rotina de manutencao...
[ATENCAO] O Roteador Central sera reiniciado em 5 minutos!
Por favor, salvem as configuracoes pendentes.
Todas as conexoes ativas serao interrompidas.
```

2. Com Parâmetros e Sem Retorno

Essas funções aceitam dados de entrada (chamados de **parâmetros** ou **argumentos**) para personalizar sua execução. Contudo, elas apenas realizam a ação e não devolvem um valor final ao programa.

Analogia de Redes:

- Pense no comando `ping 8.8.8.8`. O IP `8.8.8.8` é o parâmetro. Se você mudar o parâmetro para `192.168.1.1`, a função faz a mesma tarefa básica (enviar pacotes), mas direcionada a outro destino.
- O comando exibe as linhas na tela, mas você não armazena a resposta em uma variável do Python.

Sintaxe:

```
1 def minha_funcao(parametro):  
2     # Usa o parametro para realizar uma acao  
3     print(f"Processando dado: {parametro}")
```

Exemplo 1: Simulador de Ping

Vamos criar uma função que simula o utilitário ping (que usa pacotes ICMP – *Internet Control Message Protocol*) para testar a conectividade de um IP.

```
1 def ping(ip_destino):
2     # Simula o teste de conectividade ICMP
3     print(f"Disparando contra {ip_destino} com 32 bytes de dados:")
4     print(f"Resposta de {ip_destino}: bytes=32 tempo=12ms TTL=64")
5     print(f"Resposta de {ip_destino}: bytes=32 tempo=10ms TTL=64")
6     print(f"Estatísticas do Ping para {ip_destino}: Concluído com sucesso.")
7
8 # Utilizando a funcao com alvos diferentes:
9 ping("10.0.0.1")
10 print("-" * 30)
11 ping("google.com")
```

Exemplo 2: Bloqueio de Portas no Firewall

Função para simular o bloqueio de portas vulneráveis no Firewall do switch, registrando o motivo de segurança.

```
1 def bloquear_porta(porta, motivo):
2     # Simula comando para bloquear porta no Firewall
3     print(f"[FIREWALL] BLOQUEADO: Trafego na porta TCP {porta} suspenso.")
4     print(f"[MOTIVO] Alerta de segurança: {motivo}")
5     print("Regra aplicada com sucesso na tabela IPTables.")
6
7     # Bloqueando portas suspeitas na rede da escola
8     bloquear_porta(21, "Tentativas excessivas de login no FTP")
9     print("-" * 30)
10    bloquear_porta(23, "Protocolo Telnet nao criptografado detectado")
```

3. Sem Parâmetros e Com Retorno

Essas funções não necessitam de informações externas para iniciar. No entanto, ao finalizarem seu processamento, elas **devolvem (retornam)** um resultado para o programa principal usando a palavra-chave **return**.

Analogia de Redes:

- É como a placa de rede de um computador buscando seu próprio endereço físico (Endereço MAC – *Media Access Control*). O script da placa não precisa de entrada para descobrir seu próprio MAC, mas ela precisa devolver esse valor para o sistema operacional poder usá-lo nas comunicações.
- O valor retornado é guardado em uma variável para uso futuro.

Sintaxe:

```
1 def obter_dado():  
2     resultado = "Informacao Interna"  
3     return resultado
```

Exemplo 1: Endereço de Loopback

Vamos criar uma função que retorna o IP de loopback padrão da máquina local (127.0.0.1), usado para testes locais de rede.

```
1 def obter_ip_loopback():
2     # Retorna o IP padrao para testes locais
3     ip_local = "127.0.0.1"
4     return ip_local
5
6 # Capturando o valor de retorno em uma variavel:
7 meu_teste = obter_ip_loopback()
8
9 print(f"Iniciando conexao de teste no host: {meu_teste}")
10 # Agora podemos usar a variavel para disparar conexoes
```

Nota didática: A função `obter_ip_loopback()` não recebeu nada entre os parênteses, mas entregou um texto contendo o IP ao ser executada.

Exemplo 2: Gerador de Chave SSH Provisória

Esta função gera uma senha de acesso rápido de administrador para o SSH (geração de string com números aleatórios) para fins de teste temporário.

```
1 import random
2
3 def gerar_token_ssh():
4     # Gera uma chave numerica aleatoria para sessao SSH
5     token = random.randint(100000, 999999)
6     return token
7
8 # Solicita uma nova chave para o analista de redes:
9 chave_acesso = gerar_token_ssh()
10
11 print("--- SOLICITACAO DE CREDENCIAIS ---")
12 print(f"Sua chave SSH provisoria eh: {chave_acesso}")
13 print("Esta chave expira em 60 segundos.")
```

4. Com Parâmetros e Com Retorno

Este é o tipo mais completo e comum de função. Ela **recebe** dados de entrada para trabalhar, executa o processamento matemático ou lógico e **retorna** o resultado final para o programa.

Analogia de Redes:

- Pense em uma **Calculadora de Sub-Rede**: Você insere o endereço IP e a máscara CIDR (*Classless Inter-Domain Routing*) (parâmetros), a função realiza os cálculos binários (processamento) e te entrega a quantidade de hosts válidos na rede (retorno).
- É como um roteador decidindo rotas: entra o IP de destino → processa na tabela → retorna a interface de saída.

Sintaxe:

```
1 def calcular_soma(a, b):  
2     resultado = a + b  
3     return resultado
```

Exemplo 1: Conversor de Mbps para Kbps

Para configurar a largura de banda (*bandwidth*) de interfaces em roteadores Cisco, muitas vezes precisamos informar o valor em Kilobits por segundo (Kbps). Vamos criar uma função que converte Megabits por segundo (Mbps) para Kbps.

```
1 def converter_mbps_para_kbps(mbps):  
2     # Converte largura de banda para configuracao de interfaces  
3     kbps = mbps * 1024  
4     return kbps  
5  
6 # Configurando um link de internet de 50 Mbps:  
7 banda_config = converter_mbps_para_kbps(50)  
8  
9 print(f"Comando Cisco: bandwidth {banda_config}")  
10 # Saída: Comando Cisco: bandwidth 51200
```

Exemplo 2: Classificador de IPs (Público vs Privado)

Função que recebe um IP como string e verifica se ele pertence às faixas privadas comuns de redes locais (192.168.x.x ou 10.x.x.x), retornando True se for privado e False se for público.

```
1 def verificar_ip_privado(ip):
2     # Verifica se o IP fornecido eh da rede privada local
3     if ip.startswith("192.168.") or ip.startswith("10."):
4         return True
5     else:
6         return False
7
8 # Testando pacotes vindos da Internet e da Intranet:
9 ip_origem1 = "192.168.1.50"
10 ip_origem2 = "8.8.8.8"
11
12 print(f"0 IP {ip_origem1} eh privado? {verificar_ip_privado(ip_origem1)}")
13 print(f"0 IP {ip_origem2} eh privado? {verificar_ip_privado(ip_origem2)}")
```

Tabela Resumo dos Tipos de Funções

Aqui está o mapa mental para você nunca mais esquecer os tipos de funções no Python:

Tipo de Função	Entrada?	Saída?	Exemplo Prático (Redes)
Sem Param. e Sem Ret.	Não	Não	<code>exibir_banner()</code>
Com Param. e Sem Ret.	Sim	Não	<code>ping("10.0.0.1")</code>
Sem Param. e Com Ret.	Não	Sim	<code>obter_ip_loopback()</code>
Com Param. e Com Ret.	Sim	Sim	<code>verificar_ip_privado(ip)</code>

Tabela: Comparação dos quatro tipos de funções em Python.

Por que usar Funções? (Boas Práticas)

Como técnicos em redes, sabemos que a organização evita desastres (como cabos espalhados no rack!). No código, é a mesma coisa:

- **Evita Duplicação (Princípio DRY - Don't Repeat Yourself):** Escreva uma vez, use quantas vezes quiser.
- **Facilidade de Manutenção:** Se o cálculo da sub-rede mudar, você altera apenas dentro da função, e não em centenas de linhas de código.
- **Organização e Legibilidade:** O código fica estruturado em módulos fáceis de ler, parecendo comandos do sistema operacional.

Exercício de Fixação: Crie um script Python que automatize a verificação de uma lista de IPs da rede escolar.

```
1 # 1. Crie uma funcao que recebe um IP e retorna se ele esta ativo
2 # (Para o teste, retorne True se o IP terminar com par e False se impar)
3 def testar_host(ip):
4     ultimo_octeto = int(ip.split(".")[2])
5     if ultimo_octeto % 2 == 0:
6         return True
7     return False
8
9 # 2. Varra uma lista de IPs e use a funcao criada para exibir o status
10 ips_rede = ["192.168.1.10", "192.168.1.15", "192.168.1.20"]
11
12 for ip in ips_rede:
13     if testar_host(ip):
14         print(f"Host {ip} -> ONLINE")
15     else:
16         print(f"Host {ip} -> OFFLINE")
```

Parabéns! Você deu o primeiro passo na Automação de Redes!

Na próxima aula de Programação de Computadores, vamos integrar essas funções com a biblioteca socket para interagir com roteadores de verdade na nossa rede local!

Dúvidas?

`reginaldo.fernandes@ifce.edu.br`